claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf_b531eaef-39c\agent-a6c881cfbeb0e454a.jsonl`
- SHA-256: `7550168462dfe96e1832ed549cebce7dd1ece84ed58704261827b3ad4fdc34f8`
- Source modified: `2026-06-19T16:59:08+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_b531eaef-39c`
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`

Transcript

1. user (2026-06-19T16:57:07.033Z)

You are verifying claims for a design doc; the doc must only assert what the code supports. Verify the build-status of the levers the design reuses: (a) rally_gate.py Gate-A net-crossings ? built? eval-only or wired to prod? (b) R4 rally-END tail-trim (_trim_inactive_tail in tracknet_runner.py) ? built? (c) the shuttle-floor-contact rally-END (docs/PROMINENT_COURT_DETECTION.md §4.5) ? design-only or built? (d) shuttle_player_colocation in fusion_features.py ? built/default-OFF? Cite lines/files. For each: BUILT+WIRED / BUILT-eval-only / DESIGN-ONLY.

Read the actual files (do not guess). Return the structured verdict.

2. assistant (2026-06-19T16:57:09.162Z)

I'll verify each of the four levers by reading the actual code. Let me start by locating the relevant files.

3. user (2026-06-19T16:57:09.826Z)

backend\pipeline\detectors\rally_gate.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\tier1-windowing\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\w2-safeguard\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\hardchop\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\boxhill17\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\pr-stitch\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\gap-doc\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\resweep\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\recdoc\backend\pipeline\detectors\rally_gate.py

4. user (2026-06-19T16:57:10.293Z)

.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\tier1-windowing\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\w2-safeguard\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\hardchop\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\pr-stitch\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\boxhill17\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\gap-doc\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\resweep\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\recdoc\backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\tracknet_runner.py

5. user (2026-06-19T16:57:10.774Z)

```
backend\pipeline\detectors\fusion_features.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\tier1-windowing\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\w2-safeguard\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\hardchop\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\boxhill17\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\pr-stitch\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\gap-doc\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\resweep\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\recdoc\backend\pipeline\detectors\fusion_features.py
```

6. user (2026-06-19T16:57:11.094Z)

```
docs\PROMINENT_COURT_DETECTION.md
```

7. assistant (2026-06-19T16:57:13.720Z)

The repo root files are the ones I need (not the worktree copies). Let me read the actual production files.

8. user (2026-06-19T16:57:14.661Z)

```
1       """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3       WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4       the opponent for service after losing a point, that single trajectory looks like a
5       rally. The discriminator is structural, not duration-based: a real rally has the
6       shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7       single service feed crosses ~once.
8
9       This module is camera-agnostic and needs NO new model ? it works on the trajectory
10      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11      axis with the larger shuttle spread) and the net position (median shuttle coord on
12      that axis ? the shuttle clusters at the two ends and crosses the net region often,
13      so the median sits near the net), then counts sign changes of (coord - net) across
14      the visible points inside a candidate window, with a deadband to ignore jitter.
15
16      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18      windows from trajectory_to_action_windows, or to fold into it later.
19      """
20      from __future__ import annotations
21
22      from dataclasses import dataclass
23      from typing import List, Optional, Sequence, Tuple
24
25      Interval = Tuple[float, float]
26
27
28      @dataclass
29      class GatedWindow:
30          start: float
31          end: float
32          crossings: int
33          visible_points: int
34          kept: bool
35
36
37      def _spread(vals: Sequence[float]) -> float:
38          """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39          if len(vals) < 2:
```

```
            return 0.0
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]


    def gate_windows(points, windows: List[Interval], fps: float,
                     min_crossings: int = 2, deadband_frac: float = 0.06,
                     estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
        """Annotate each (start,end) window with its shuttle net-crossing count and a
        keep decision (crossings >= min_crossings). Deadband is a fraction of the play
        axis span. Net/axis are estimated once over the whole trajectory (stable).

        `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
        `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
        per window ? same result, avoids the per-window re-estimation."""
        axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
        deadband = deadband_frac * span
```

```
        out: List[GatedWindow] = []
        for (s, e) in windows:
            wp = _points_in(points, fps, s, e)
            c = count_net_crossings(wp, axis, net, deadband)
            vis = sum(1 for p in wp if p.visible)
            out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
                                   kept=c >= min_crossings))
        return out


    def apply_gate(points, windows: List[Interval], fps: float,
                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
        """Convenience: return only the windows that pass the crossing gate."""
        return [(g.start, g.end) for g in
                gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]

```

9. user (2026-06-19T16:57:14.915Z)

```
      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).

      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
      math is unit-testable in a GPU-free dev container. The detection (person boxes,
      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
      module only turns those signals into a handful of **scale/translation-invariant**
      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
      memorise this court/camera and fail to generalise off a small labelled set).

      Conventions:
      - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
needs it.
      - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
      - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
      - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
every
        detection counts ? the player-count-only mode).
      - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).

      Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
      floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
      """
      from __future__ import annotations

      from typing import Dict, List, Optional, Sequence, Tuple

      BBox = Tuple[float, float, float, float]
      Net = Tuple[str, float]


      def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
          """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
          (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
          if polygon is None or len(polygon) < 3:
              return False
          x, y = point
          inside = False
          n = len(polygon)
          j = n - 1
```

```
        for i in range(n):
            xi, yi = polygon[i]
            xj, yj = polygon[j]
            # Does the horizontal ray at y cross edge (i, j)?
            if (yi > y) != (yj > y):
                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
                if x < x_cross:
                    inside = not inside
            j = i
        return inside


    def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        """Normalised foot point (bottom-centre) of a person box ? where the player stands,
        the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, _y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)


    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
        if frame_width <= 0 or frame_height <= 0:
            return (0.0, 0.0)
        x1, y1, x2, y2 = box
        return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)


    def person_in_court(box: BBox, frame_width: float, frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
        """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
        True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
        point meaningless, so a masked check returns False rather than testing a bogus
(0,0)."""
        if court_polygon is None:
            return True
        if frame_width <= 0 or frame_height <= 0:
            return False
        return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)


    def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                        frame_height: float,
                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
        """Per-frame count of in-court persons (one entry per sampled frame)."""
        return [
            sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
            for _frame_idx, dets in sorted(per_frame.items())
        ]


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))


    def players_in_court(counts: Sequence[int]) -> float:
        """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
```

```
        return _median(list(counts))


    def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
        """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
        if not counts:
            return 0.0
        return sum(1 for c in counts if c >= min_players) / len(counts)


    def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
                           frame_width: float, frame_height: float) -> float:
        """Mean per-track centre displacement between consecutive sampled frames, with x
        normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
        not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
        if frame_width <= 0 or len(per_frame) < 2:
            return 0.0
        frames = sorted(per_frame.keys())
        steps: List[float] = []
        for a, b in zip(frames, frames[1:]):
            prev = {tid: box for tid, box in per_frame[a]}
            for tid, box in per_frame[b]:
                if tid in prev:
                    cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
                    cx1, cy1 = _center_norm(box, frame_width, frame_height)
                    steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
        if not steps:
            return 0.0
        return sum(steps) / len(steps)


    def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
                         frame_height: float, net: Optional[Net],
                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
        """Fraction of sampled frames with >=1 in-court person on BOTH net halves.

        A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
        The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
        """
        if net is None or not per_frame:
            return 0.0
        axis, pos = net
        both = 0
        for _frame_idx, dets in per_frame.items():
            near, far = False, False
            for _tid, box in dets:
                if not person_in_court(box, frame_width, frame_height, court_polygon):
                    continue
                fx, fy = _foot_norm(box, frame_width, frame_height)
                coord = fx if axis == "x" else fy
                if coord < pos:
                    near = True
                else:
                    far = True
            if near and far:
                both += 1
        return both / len(per_frame)


    def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
                        pad_frac: float) -> bool:
        """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
```

```
161         width/height (the 'adjacent / in-hand' tolerance)?"""
162         x1, y1, x2, y2 = box
163         px = pad_frac * frame_width
164         py = pad_frac * frame_height
165         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                                   frame_width: float, frame_height: float,
170                                   pad_frac: float = 0.03) -> float:
171         """Fraction of (visible shuttle frames that were also person-sampled) in which the
172         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
173         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
174         """
175         if not per_frame or frame_width <= 0 or frame_height <= 0:
176             return 0.0
177         aligned = 0
178         colocated = 0
179         for p in shuttle_points or []:
180             if not getattr(p, "visible", False):
181                 continue
182             dets = per_frame.get(int(p.frame))
183             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
184                 continue
185             aligned += 1
186             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187                    for _tid, box in dets):
188                 colocated += 1
189         return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                                 frame_width: float, frame_height: float,
202                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                                 net: Optional[Net] = None,
204                                 min_players: int = 2) -> Dict[str, float]:
205         """Aggregate the per-window person features from per-frame detections + the in-
window
206         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208         return {
209             "players_in_court": players_in_court(counts),
210             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216
```

10. assistant (2026-06-19T16:57:18.189Z)

Now let me check whether `rally_gate` / `apply_gate` / `gate_windows` is wired into production (called outside tests/eval), and read the tracknet_runner for the tail-trim.

11. user (2026-06-19T16:57:20.311Z)

```
Found 38 files
artifacts\pr245_review.json
artifacts\pr245_fixes_plan.md
artifacts\pr245_code_review.md
artifacts\pr245_review.md
docs\PROMINENT_COURT_DETECTION.md
docs\README.md
docs\RALLY_QUALITY_RESEARCH.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\QUALITY_ITERATIONS.md
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\CODE_MAP.md
CLAUDE.md
backend\eval\rally_seg_eval.py
backend\pipeline\detectors\court_gate.py
tests\test_rally_seg_eval.py
backend\eval\experiment.py
backend\config\models.py
tests\test_served_gate_a_regression.py
docs\GOLDEN_REGRESSION_FIXTURES.md
backend\pipeline\segmenters\fusion_hybrid.py
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\ANALYZERS_AND_RECONCILERS.md
docs\COMMERCIALIZATION.md
backend\eval\serve_contrast.py
backend\eval\distill_local.py
backend\eval\fusion_golden.py
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
tests\test_rally_gate.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\eval\calibrate_local.py
backend\pipeline\detectors\__init__.py
backend\eval\eval_partial_labels.py
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
backend\eval\export_wasb_segments.py
```

12. user (2026-06-19T16:57:21.659Z)

```
Found 25 files
docs\PROMINENT_COURT_DETECTION.md
docs\README.md
docs\RALLY_QUALITY_RESEARCH.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\QUALITY_ITERATIONS.md
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\CODE_MAP.md
backend\pipeline\detectors\court_gate.py
backend\eval\fusion_compare.py
backend\eval\fusion_audio.py
backend\eval\ablation.py
tests\test_fusion_compare.py
tests\test_audio_features.py
tests\test_fusion_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
```

```
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
docs\ANALYZERS_AND_RECONCILERS.md
docs\EXPERIMENT_HARNESS.md
backend\eval\fusion_golden.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\detectors\audio_features.py
tests\test_fusion_features.py
backend\pipeline\detectors\fusion_features.py
```

13. user (2026-06-19T16:57:22.789Z)

```
Found 8 files
artifacts\pr245_review.json
artifacts\pr245_fixes_plan.md
artifacts\pr245_code_review.md
artifacts\pr245_review.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\QUALITY_ITERATIONS.md
backend\pipeline\detectors\tracknet_runner.py
tests\test_tracknet_runner.py
```

14. assistant (2026-06-19T16:57:26.968Z)

Key question: is `rally_gate` wired into the **production segmenter** (`fusion_hybrid.py`), or only into `backend/eval/`? Let me look at the production segmenter and the `__init__.py`, plus the tracknet_runner tail-trim. The `fusion_hybrid.py` matched both `rally_gate` and `fusion_features` searches, so I need to check whether those are actually invoked there or just imported/referenced.

15. user (2026-06-19T16:57:27.926Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
```

```
    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift through the
    experiment harness (EXPERIMENT_HARNESS.md), never silently.
    """
    from typing import Any, Dict, List, Optional, Tuple

    from backend.pipeline.segmenters.base import segmenter_registry
    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
    from backend.pipeline.detectors.base import DetectorRunner
    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
    from backend.pipeline.detectors import rally_gate
    from backend.pipeline.detectors import fusion_features as ff


    class FusionSegmenter(TrajectoryHybridSegmenter):
        """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate B)."""

        family = "fusion"
        display_label = "Fusion"

        def __init__(self, db: Any, config: Any):
            super().__init__(db, config)
            # Built lazily only if the person cue is enabled (no detector model loaded otherwise).
            self._person: Optional[Any] = None
            # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-estimate it
            # over the whole trajectory once per candidate window (it depends only on `points`).
            self._axis_cache_key: Optional[int] = None
            self._axis_cache: Optional[tuple] = None

        @property
        def name(self) -> str:
            return "fusion_hybrid"

        @property
        def description(self) -> str:
            return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate rallies, "
                    "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and the "
                    "AI provider sets semantic boundaries.")

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
            substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
            if substrate == "tracknet":
                cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
                return TrackNetRunner(cfg), {
                    "weights_path": cfg.weights_path,
                    "queue_length": cfg.queue_length,
                    "fusion_substrate": "tracknet",
                }
            wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate": "wasb"}

        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            stats = super()._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
```

```
            # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
            # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
            # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
            gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
                                            min_crossings=0,
estimate=self._axis_estimate(points))
            stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

            fcfg = idx_cfg.get("fusion", {})
            if fcfg.get("cue_flags", {}).get("person", False):
                stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
            return stats

        # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
        # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
        # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
        # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
        # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
        # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
        # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.

        def _axis_estimate(self, points: List[Any]) -> tuple:
            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
            computed once per video, not once per candidate window (it depends only on
points)."""
            key = id(points)
            if self._axis_cache_key != key or self._axis_cache is None:
                self._axis_cache_key = key
                self._axis_cache = rally_gate.estimate_play_axis(points)
            est = self._axis_cache
            assert est is not None
            return est

        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                             frame_width: float, video_path: str,
                             fcfg: Dict[str, Any]) -> Dict[str, float]:
            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
            frame range and compute the scale/translation-invariant person features. Lazily
builds
            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
            from backend.pipeline.detectors.person_detector import PersonDetector
            if self._person is None:
                self._person = PersonDetector(self.config)

            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
```

```
127             fw = det["frame_width"] or frame_width
128             fh = det["frame_height"]
129             # If we can't trust the frame dims, every normalised feature would silently
collapse
130             # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131             if fw <= 0 or fh <= 0:
132                 return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134             poly = fcfg.get("court_polygon")
135             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
136             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
139                                               court_polygon=polygon, net=net)
140
141         def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                                 window_stats: List[Dict[str, Any]],
143                                 idx_cfg: Dict[str, Any]) -> List[int]:
144             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145             (0) keep every window ? identical to the substrate. Persisted candidates are
146             untouched (full superset).
147
148             NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
149             thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
150             where no segment clears ``min_crossings``), the gate falls back to the full
ungated
151             set rather than emit a video with zero rallies. This makes Gate A/B strictly
152             *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
153             the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
154             below it always wins on an otherwise-empty video. Observed on real footage
155             (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156             gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157             fcfg = idx_cfg.get("fusion", {})
158             min_crossings = int(fcfg.get("min_crossings", 0))
159             min_players = int(fcfg.get("min_players", 0))
160             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162             keep: List[int] = []
163             for i, st in enumerate(window_stats):
164                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                     continue
166                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                     continue
168                 keep.append(i)
169
170             if not keep and window_stats:
171                 # Gating emptied a non-empty video ? fall back to the ungated set.
172                 return list(range(len(window_stats)))
173             return keep
174
175
176     segmenter_registry.register(FusionSegmenter)
177
```

16. user (2026-06-19T16:57:28.598Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
```

```
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False                       # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)
44
45          @classmethod
46          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47              tn = (idx_cfg or {}).get("tracknet", {}) or {}
48              return cls(
49                  distro=tn.get("wsl_distro", "Ubuntu"),
50                  repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51                  conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52                  conda_env=tn.get("conda_env", "TrackNetV4"),
53                  weights_path=tn.get("weights_path", ""),
54                  queue_length=int(tn.get("queue_length", 5)),
55                  stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56                  wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57                  timeout_sec=int(tn.get("timeout_sec", 1800)),
58                  keep_frames=bool(tn.get("keep_frames", False)),
```

```
                min_free_gb=float(tn.get("min_free_gb", 0.0)),
            )


    def to_wsl_mnt_path(win_path: str) -> str:
        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

        Already-POSIX paths (starting with / or ~) are returned unchanged so the
        same helper is safe to call on either kind of path.
        """
        p = str(win_path)
        if p.startswith("/") or p.startswith("~"):
            return p
        p = p.replace("\\", "/")
        if len(p) >= 2 and p[1] == ":":
            drive = p[0].lower()
            return f"/mnt/{drive}{p[2:]}"
        return p


    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner(WslCommandMixin, DetectorRunner):
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

        Implements the common DetectorRunner contract so a future Linux-native or
        alternative trajectory runner can be substituted without touching the hybrids.
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
```

```
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
            therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
            a filename cannot collide on the output CSV.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
            # relative paths through unchanged, so WSL would resolve them against the
            # predict.py cwd instead of the repo (same bug class as wasb_runner).
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            # Namespace by video_id so two same-filename videos don't collide on the CSV.
            # predict.py derives its output name from the (staged) video stem, so staging
            # under the namespaced name propagates into "<key>_predict.csv".
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, f"{key}{ext}")
                if staged is None:
                    return None
                wsl_video = staged
            else:
                # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
                # We cannot rename its output, so fall back to the un-namespaced name.
                wsl_video = to_wsl_mnt_path(video_win_path)
                expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            if self.cfg.min_free_gb > 0:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
                                   "consider running tools/cleanup_caches.py.",
                                   free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"cd {self.cfg.repo_dir}/src && "
                f"python predict.py "
                f"--video_path {wsl_tilde_quote(wsl_video)} "
                f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
```

```
185                 f"--output_dir {wsl_tilde_quote(wsl_out)} "
186                 f"--queue_length {self.cfg.queue_length}"
187             )
188             logger.info("Running TrackNet inference on %s ...", base)
189             try:
190                 res = self._wsl_bash(cmd)
191             except subprocess.TimeoutExpired:
192                 logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
193                 return None
194
195             if res.returncode != 0:
196                 logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197                 return None
198
199             if not os.path.exists(expected_csv_win):
200                 logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201                 return None
202             logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204             # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205             # regenerable intermediate). On earlier failure we return above without
cleaning.
206             if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207                 logger.info("Cache hygiene: removing staged video for %s "
208                             "(set indexing.tracknet.keep_frames=true to retain).", key)
209                 self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212         def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213             """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215             Returns the WSL path to the staged copy, or None on failure.
216             """
217             src_mnt = to_wsl_mnt_path(video_win_path)
218             reason = self.wsl_source_unreadable_reason(src_mnt)
219             if reason:
220                 logger.error("Cannot stage video into WSL: %s", reason)
221                 return None
222             dest = f"{self.cfg.wsl_stage_dir}/{base}"
223             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224             try:
225                 res = self._wsl_bash(cmd)
226             except subprocess.TimeoutExpired:
227                 logger.error("Staging video into WSL timed out.")
228                 return None
229             if res.returncode != 0 or "OK" not in (res.stdout or ""):
230                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231                 return None
232             return dest
233
234         # ------------------------------------------------------------ CSV -> windows
235
236         @staticmethod
237         def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238             """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239             points: List[TrajectoryPoint] = []
240             with open(csv_win_path, newline="") as f:
241                 reader = csv.DictReader(f)
242                 for row in reader:
243                     try:
```

```
                        points.append(TrajectoryPoint(
                            frame=int(row["Frame"]),
                            visible=int(row["Visibility"]) == 1,
                            x=float(row["X"]),
                            y=float(row["Y"]),
                        ))
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
                           velocity_thresh: float) -> List[Tuple[int, float]]:
            """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.

            Computes each visible point's normalised velocity from the last *visible* point;
            an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
            ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
            the caller so identical floats feed the velocity math."""
            active = []  # (frame_idx, velocity)
            prev: Optional[TrajectoryPoint] = None
            for p in points:
                if not p.visible:
                    prev = None  # break the trajectory; absent shuttle = not in flight
                    continue
                if prev is not None:
                    dframes = p.frame - prev.frame
                    if dframes > 0:
                        dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                        velocity = (dist / frame_width) * (fps / dframes)
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
                prev = p
            return active

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
            max_window_duration: float = 0.0,
            min_split_gap: float = 0.5,
            window_hard_cap: bool = False,
            window_inactivity_end: float = 0.0,
            inactivity_min_density: float = 0.34,
            inactivity_rally_thresh: float = 0.08,
            window_blob_fallback: bool = False,
            window_blob_factor: float = 4.0,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
```

```
306             shape feeding the AI-handoff phase is identical.
307
308             ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
309             the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
310             and a single window swallows several rallies + the dead time between them. When
set, any
311             window longer than this is recursively SPLIT at its largest internal inactivity
gap (the
312             longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
313             that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
314             ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
315             drop, and it is config-gated default-OFF until measured on the golden set.
316
317             ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
318             over-long window has NO internal inactivity gap to split on (a continuously-
active
319             trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
320             blob where the gap-splitter alone left a single 174 s window), it is recursively
321             hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
322             (never merges), so recall cannot drop; gated default-OFF until measured.
323
324             ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
325             ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
326             over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
327             force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` ×
328             ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329             their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
330             alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331             each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332             rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333             can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334             """
335             if fps <= 0:
336                 fps = 30.0
337             if frame_width <= 0:
338                 frame_width = 1280.0
339
340             active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342             if not active:
343                 return []
344
345             max_gap_frames = int(merge_gap * fps)
346
347             def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348                 vels = [v for _, v in group]
```

```
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(vels),
                    "mean_velocity": sum(vels) / len(vels),
                    "track_id_count": 1,  # single shuttle; kept for window-shape parity
                    "chunk_index": chunk_index,
                }

            groups: List[List[Tuple[int, float]]] = []
            group = [active[0]]
            for af in active[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    groups.append(group)
                    group = [af]
            groups.append(group)

            if max_window_duration > 0:
                groups = TrackNetRunner._split_overlong_groups(
                    groups, int(max_window_duration * fps), int(min_split_gap * fps),
                    hard_cap=window_hard_cap)

            # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
            # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
            # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
            # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
            # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
            if window_inactivity_end > 0:
                tw = int(window_inactivity_end * fps)
                groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
                                                             inactivity_rally_thresh) for g
in groups]

            # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
            # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
            # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
            # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
            forced_flags = [False] * len(groups)
            if window_blob_fallback and max_window_duration > 0:
                groups, forced_flags = TrackNetRunner._blob_fallback_chop(
                    groups, int(max_window_duration * fps), window_blob_factor)

            windows = []
            for g, forced in zip(groups, forced_flags):
                w = _finalize(g)
                if forced:
                    w["forced_cut"] = True
                windows.append(w)
            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

        @staticmethod
        def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
                                min_density: float, rally_thresh: float) -> List[Tuple[int,
```

```
float]]:
404             """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
405             (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
406             over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
407             (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
408             ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
409             LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
410             can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
411             exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
412             is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
413             if trail_frames <= 0 or len(group) < 2:
414                 return group
415             frames = [f for f, _ in group]
416             fast = [1 if v > rally_thresh else 0 for _, v in group]
417             lo = 0
418             fast_sum = 0
419             last_dense = -1
420             for i in range(len(frames)):
421                 fast_sum += fast[i]
422                 while frames[lo] <= frames[i] - trail_frames:
423                     fast_sum -= fast[lo]
424                     lo += 1
425                 win_count = i - lo + 1
426                 if win_count > 0 and fast_sum / win_count >= min_density:
427                     last_dense = i
428             return group[: last_dense + 1] if last_dense >= 0 else group
429
430         @staticmethod
431         def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
432             """Index of the active frame nearest the group's temporal midpoint (used to chop
a
433             gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
fallback."""
434             mid = (group[0][0] + group[-1][0]) / 2.0
435             split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
436             return split_i
437
438         @staticmethod
439         def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
440                                    min_split_frames: int,
441                                    hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
442             """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
443             internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
444             boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
445             invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
446             rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
447             temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
448             bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
449             if max_win_frames <= 0:
```

```
450                 return groups
451             out: List[List[Tuple[int, float]]] = []
452             work = list(groups)
453             while work:
454                 g = work.pop()
455                 if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
456                     out.append(g)
457                     continue
458                 best_i, best_gap = -1, -1
459                 for i in range(1, len(g)):
460                     gap = g[i][0] - g[i - 1][0]
461                     if gap > best_gap:
462                         best_gap, best_i = gap, i
463                 if best_i < 0 or best_gap < min_split_frames:
464                     # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
465                     # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
466                     if hard_cap:
467                         split_i = TrackNetRunner._midpoint_split_index(g)
468                         work.append(g[:split_i])
469                         work.append(g[split_i:])
470                     else:
471                         out.append(g)
472                     continue
473                 work.append(g[:best_i])
474                 work.append(g[best_i:])
475             out.sort(key=lambda grp: grp[0][0])
476             return out
477 
478         @staticmethod
479         def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
480                                 blob_factor: float = 4.0
481                                 ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482             """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483             inactivity trim have each had their chance, a group still longer than
``blob_factor`` ×
484             ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
485             signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
486             11× a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2× the cap)
487             is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
488             midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
489             degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
490             rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
491             can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
492             that piece came from a forced chop. Iterative worklist to bound stack depth."""
493             if max_win_frames <= 0:
494                 return groups, [False] * len(groups)
495             blob_frames = max_win_frames * max(1.0, blob_factor)
496             out: List[List[Tuple[int, float]]] = []
497             flags: List[bool] = []
498             forced = 0
499             for g in groups:
500                 if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
501                     out.append(g)
502                     flags.append(False)
503                     continue
```

```
504                 pieces: List[List[Tuple[int, float]]] = []
505                 work = [g]
506                 while work:
507                     h = work.pop()
508                     if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
509                         pieces.append(h)
510                         continue
511                     split_i = TrackNetRunner._midpoint_split_index(h)
512                     work.append(h[:split_i])
513                     work.append(h[split_i:])
514                 pieces.sort(key=lambda grp: grp[0][0])
515                 out.extend(pieces)
516                 flags.extend([True] * len(pieces))
517                 forced += len(pieces) - 1
518             if forced:
519                 logger.warning(
520                     "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
521                     "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
522                     "motion), not a bigger cap", forced)
523             return out, flags
524
```

17. assistant (2026-06-19T16:57:37.027Z)

Now I have the full picture on (a), (b), and (d). Key findings:

- **(a) rally_gate Gate-A**: `gate_windows`/`estimate_play_axis` ARE invoked in production `fusion_hybrid.py` (lines 83-84, 106) to persist `net_crossings`, and `_select_for_handoff` (lines 141-173) can gate on `min_crossings`. BUT `fusion_hybrid` is NOT the default segmenter (line 22-26: "Registered as `fusion_hybrid`, NOT the default segmenter ? it is opt-in"). And gating is default-OFF (`min_crossings` default 0). I need to confirm whether `fusion_hybrid` is the default segmenter or not, and the config defaults.

- **(b) `_trim_inactive_tail`**: BUILT (lines 401-428), invoked in `trajectory_to_action_windows` (lines 379-382) gated on `window_inactivity_end > 0`, default 0.0 (OFF) per line 294.

- **(d) `shuttle_player_colocation`**: BUILT in fusion_features.py (lines 168-189), in `PERSON_FEATURE_NAMES`, computed only when person cue is ON (default OFF per line 88, 160).

Now let me verify (c) the shuttle-floor-contact rally-END design, the config defaults, and the default segmenter. Let me read the relevant doc section and config.

18. user (2026-06-19T16:57:37.978Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
```

```
        court_green_lower: list[int] = Field(default=[35, 40, 40])
        court_green_upper: list[int] = Field(default=[85, 255, 255])
        court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


    class ValidationConfig(BaseModel):
        min_resolution_width: int = 1280
        min_resolution_height: int = 720
        min_fps: float = 29.9
        min_blur_threshold: float = 20.0
        max_scene_cuts_per_minute: float = 0.5
        relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)


    class TrackNetConfig(BaseModel):
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/TrackNetV4"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "TrackNetV4"
        weights_path: str = ""
        queue_length: int = 5
        stage_in_wsl: bool = True
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
        # decoded frame cache) are deleted automatically ? they are pure intermediates,
        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
        # 0 = disabled.
        min_free_gb: float = 0.0


    class WasbIndexConfig(BaseModel):
        """Typed config for the live WASB shuttle substrate (indexing.wasb).

        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
        these knobs actually take effect ? previously this whole block was silently dropped
        because nested config sections defaulted to extra='ignore'.
        """
        wsl_distro: str = "Ubuntu"
        repo_dir: str = "~/models/WASB-SBDT"
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "wasb"
        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
        sport: str = "badminton"
        wsl_stage_dir: str = "~/clips"
        timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
        velocity_threshold: float = 0.02
        # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
        # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
        # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
        keep_frames: bool = False
        # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
        min_free_gb: float = 0.0
        # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
        # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
```

```
78          # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
79          # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
80          # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
81          stream_video: bool = False
82
83
84      class FusionConfig(BaseModel):
85          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
86
87          Every default reproduces control behaviour: the fusion segmenter persists the
88          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
89          true ? the person-cue features, but it does NOT gate the served handoff unless a
90          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
91          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
92          supplied ? off-court persons (spectators / adjacent courts) are excluded.
93          """
94          # Which trajectory substrate seeds the windows (shuttle stays primary).
95          substrate: Literal["wasb", "tracknet"] = "wasb"
96          # Windowing velocity threshold for the fusion family (the engine reads
97          # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98          # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99          velocity_threshold: float = 0.02
100         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
101         # enabled ? so the default path never imports torch / touches the GPU.
102         cue_flags: dict[str, bool] = Field(default_factory=dict)
103         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
104         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
105         min_crossings: int = Field(default=0, ge=0)
106         # Served Gate B: when the person cue is on, drop windows with median in-court
players
107         # < this. 0 = OFF.
108         min_players: int = Field(default=0, ge=0)
109         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
110         court_polygon: Optional[list[list[float]]] = None
111         # Net line for the person two-sided-motion split (person features only ? the shuttle
112         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
113         net_axis: Literal["x", "y"] = "y"
114         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
115
116
117     class IndexingConfig(BaseModel):
118         default_segmenter: str = "yolo_hybrid"
119         use_gpu: bool = True
120         motion_threshold: float = 0.08
121         min_segment_duration: float = 3.0
122         dead_time_merge_gap: float = 2.0
123         chunk_duration: float = 90.0
124         chunk_overlap: float = 10.0
125         detector_model: str = "fasterrcnn_resnet50_fpn"
126         detector_score_threshold: float = 0.5
127         yolo_velocity_threshold: float = 0.15
128         yolo_frame_skip: int = 3
129         yolo_tracker: str = "bytetrack.yaml"
130         yolo_prefetch_workers: int = 2
131         min_action_window_duration: float = 2.0
132         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
```

```
133         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
134         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
135         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
136         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
137         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
138         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
139         max_window_duration: float = 6.0
140         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
141         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
142         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
143         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
144         # Only cuts (recall can't drop). OFF by default until measured/promoted.
145         window_hard_cap: bool = False
146         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
147         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
148         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
149         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
150         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
151         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
152         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
153         window_inactivity_end: float = 1.5
154         inactivity_rally_thresh: float = 0.20
155         inactivity_min_density: float = 0.30
156         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
157         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
158         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
159         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
160         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
161         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
162         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
163         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
164         window_blob_fallback: bool = False
165         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
166         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
167         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
168         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
169         window_blob_factor: float = 4.0
```

```
        log_yolo_candidates: bool = True
        store_yolo_candidates: bool = True
        ai_handoff_padding: float = 2.0
        ai_max_workers: int = 5
        # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
        # chunks of a high-bitrate (4K) source blow past the provider upload cap
        # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
        # Matches gemini_refine's --clip-scale-width default.
        ai_chunk_scale_width: int = 1280
        # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
        # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
        # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
        # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
        # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
        skip_ai_handoff: bool = False
        # Optional debug knob: trim every video to the first N seconds before processing.
        debug_duration: Optional[float] = None

        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
        fusion: FusionConfig = Field(default_factory=FusionConfig)

        @field_validator("default_segmenter")
        @classmethod
        def segmenter_must_be_registered(cls, v: str) -> str:
            # Registry check happens at runtime in main/segmenter selection.
            # Here we just allow any string for forward compatibility.
            return v

        @model_validator(mode="after")
        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
            # step makes `while t < duration: t += step` loop forever, growing memory before
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
```

```
228         text: str = "Generated by Khelsutra.guru"
229         logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
230         font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
231         font: str = "Sans"         # fontconfig family used when font_path is empty
232         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
233         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
234         box: bool = True           # faint backing box behind the text for legibility
235         margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
236         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
237
238
239     class StitchingConfig(BaseModel):
240         begin_padding: float = 0.5
241         end_padding: float = 0.5
242         use_cache: bool = False
243         min_shot_count: int = 1
244         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
245         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
246         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
247         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
248         # local detector over-fires and its false positives are mostly SHORT (motion blips,
249         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
250         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
251         min_rally_duration: float = Field(default=0.0, ge=0.0)
252         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
253
254
255     class LoggingConfig(BaseModel):
256         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
257         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
258         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
259         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
260         # them to WARNING; set true to restore full chatter for request-flow debugging.
261         verbose_http: bool = False
262
263
264     class SecurityConfig(BaseModel):
265         # When set, /api/process video_path must resolve under this directory (hosted/tenant
266         # mode). Default None preserves the single-user local 'any local file' workflow.
267         allowed_video_root: Optional[str] = None
268
269         # --- Chunked upload (B2, PR-2) ---
270         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
271         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
272         # contain upload_dir or /api/process will reject the uploaded paths.
273         upload_dir: str = "output/uploads"
274         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
275         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
276         max_upload_mb: int = 4096
277         max_part_mb: int = 95
278         allowed_upload_extensions: List[str] = ["mp4", "mov"]
279
280
281     class AppConfig(BaseModel):
282         """Root configuration object.
283
284         All runtime configuration should be accessed via an instance of this class
```

```
        (or the loader) rather than raw dict lookups.
        """

        sport: Sport = "badminton"
        ai_provider: str = "gemini"
        # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
        # the alias never deprecates underneath us. Eval reports record the run date
alongside.
        ai_model: str = "gemini-flash-latest"

        validation: ValidationConfig = Field(default_factory=ValidationConfig)
        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
        output: OutputConfig = Field(default_factory=OutputConfig)
        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
        security: SecurityConfig = Field(default_factory=SecurityConfig)
        logging: LoggingConfig = Field(default_factory=LoggingConfig)

        # Allow extra keys during transition so existing config.json files do not break.
        model_config = {
            "extra": "allow",
            "validate_assignment": True,
        }

        # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
        # Will be removed once all call sites (validators, segmenters, etc.) move to typed
        # attribute access / the typed config. A nested *section* is returned as a plain
dict
        # via model_dump, so legacy chained access keeps working ? returning the Pydantic
        # section model instead breaks it (a model has no .get), which bit every validator
on
        # real runs. Scalar leaves are returned as-is.
        def get(self, key: str, default: Any = None) -> Any:
            _MISSING = object()
            val: Any = _MISSING
            if key in type(self).model_fields or hasattr(self, key):
                val = getattr(self, key, _MISSING)
            if val is _MISSING:
                # Legacy convenience: find a leaf key inside any nested section. Sections
are
                # discovered dynamically from the model fields, so a future top-level
section
                # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
                for fname in type(self).model_fields:
                    section = getattr(self, fname, None)
                    if isinstance(section, BaseModel) and hasattr(section, key):
                        val = getattr(section, key)
                        break
            if val is _MISSING:
                return default
            if isinstance(val, BaseModel):
                return val.model_dump(mode="json")
            return val

        def __getitem__(self, key: str) -> Any:
            val = self.get(key)
            if val is None and key not in type(self).model_fields:
                # Re-raise like a real dict for unknown top-level
                raise KeyError(key)
            return val


    # Convenience type alias for callers that want to annotate
```

```
342     Config = AppConfig
343
```

19. user (2026-06-19T16:57:39.282Z)

```
Found 13 files
backend\pipeline\detectors\court_gate.py
backend\main.py
backend\infrastructure\database.py
backend\eval\golden_real_fixtures.py
backend\eval\golden_fixtures.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_infer.py
backend\eval\serve_contrast.py
backend\eval\promotion.py
backend\pipeline\personalization\serving.py
backend\eval\per_user_gate.py
backend\eval\eval_stats.py
backend\api\static\app.js
```

20. assistant (2026-06-19T16:57:46.178Z)

Important refinements. Config shows:
- `default_segmenter: str = "yolo_hybrid"` (line 118) ? NOT fusion_hybrid. So the Gate-A rally_gate path lives in the `fusion_hybrid` segmenter, which is not the default.
- `window_inactivity_end: float = 1.5` (line 153) ? R4 tail-trim is **DEFAULT-ON** now (the R1 milestone flip), not OFF. The tracknet_runner.py docstring saying "default OFF" is the function-level default; the production config default is 1.5 (ON). This is a key nuance.

Now I need to check the floor-contact grep hits in tracknet_runner.py and court_gate.py to see if floor-contact rally-END is actually implemented, and read the doc §4.5. The grep for "floor" hit tracknet_runner.py ? let me check what that match is (it may just be the word "floor" in a comment or coincidental). Let me search more precisely and read the doc section.

21. user (2026-06-19T16:57:48.182Z)

```
445:        invent or drop coverage; a group with no gap ? the floor is left intact (a genuinely
long
```

22. user (2026-06-19T16:57:49.113Z)

```
17:to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is still
31:    """Is the normalised point inside the court floor polygon, OR within ``height_pad`` above
it?
33:    The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the floor)
and
34:    finding it inside means the original sits within ``height_pad`` above the floor ? a high
shuttle
```

23. user (2026-06-19T16:57:49.723Z)

```
1       # Prominent-Court Detection ? kill background-court false positives (multicourt)
2
3       > **Status:** `DRAFT` ? design + experiment **owner-gated for any default flip** ·
**Owner:** Avi · **Created:** 2026-06-19 · **Last updated:** 2026-06-19
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5       > **Tracking anchors:** §8 progress tracker is the source of truth; pointer in memory
`current-state`.
6       > **Relation to existing docs:** attacks gap **G2** ("precision ? local over-fires,
worst on multicourt") from [`RALLY_DETECTION_GAP_CLOSURE.md`](RALLY_DETECTION_GAP_CLOSURE.md);
it is the **shuttle-side** counterpart of the person-side **Gate B / court mask** in
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md). Complements the held-shuttle
**`shuttle_player_colocation`** (G3) ? court-localization (G2) and held-shuttle (G3) are
```

independent precision levers.
7   > **Honesty note:** claims are tagged `[verified]` (checked vs code/measured) / `[design]` (proposed). Every signal lands **flag-gated, default-OFF**, promoted only on measured lift per [`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md) + [[decision-rigor-norm]].
8
9   ---
10
11   ## 0. TL;DR
12   In **multicourt** footage the camera sees several courts. The shuttle detector (WASB) tracks *any* shuttle in frame, so a rally on a **background / adjacent court** becomes a predicted rally on the **main match** ? a false positive. The fix: identify the **prominent court** (the foreground court that occupies the majority of the frame, where the match being filmed is played) and **keep only rallies whose shuttle lives inside it**. Mechanically this re-defines the shuttle signal from `(x, y, visible)` to `(x, y, visible-AND-inside-the-prominent-court)` *before* windowing ? so a background-court shuttle reads as "not there" and never forms a rally. A flat 2D floor polygon is the MVP; a **pseudo-3D play-volume** (court floor + the air column above it) is the refinement so a high clear over the prominent court isn't wrongly rejected. Lands as a **default-OFF experiment**; the flip is gated on a Launch Report ([[launch-report-convention]]).
13
14   ## 1. Problem & motivating example `[verified]`
15   - **Measured gap (G2):** local over-fires ~2× vs Gemini, and the over-fire is **worst on multicourt** ? even among long (>7s) rallies, multicourt over-fires **1.58×** (background-court games are *real* long rallies, just on the wrong court). See `RALLY_DETECTION_GAP_CLOSURE.md` §3 + the P7 long-rally lens.
16   - **Concrete anchor ? GX010142 rally #1 (2026-06-19).** Owner observation: *"the first rally is just a shuttle flying in the non-prominent court? while the prominent-court players have not started."* Confirmed from the cached WASB trajectory (`output/wasb/GX010142_1080p__*_wasb.csv`): rally #1 (`0.72?3.65s`, 2.94s) has a shuttle centroid at **meanX ? 1616** on a 1920-wide frame (?84% to the right) ? spatially apart from the central cluster where most rallies live (meanX ? 1000?1300). Several other suspicious rallies (#3, #9, #12, #16) cluster in the same far-right band ? consistent with a **right-side adjacent court**.
17   - **Why a 1D threshold is not enough `[verified]`:** the courts do not separate cleanly on X or Y alone (rally Y-centroids overlap 250?620 px across courts; the far-right band mixes with foreground play). The courts are **2D regions** under perspective ? so we need the court *geometry*, not a coordinate cutoff. This is the owner's "pseudo-3D / court model" intuition, validated by the data.
18
19   ## 2. Goal & non-goals
20   - **Goal:** on multicourt clips, drop rallies whose shuttle is outside the prominent court, **without losing real prominent-court rallies** ? measured per-video, no regression on single-court clips (where it must be a no-op or a do-no-harm).
21   - **Non-goals (v1):** multi-camera; tracking players across courts; full court-line homography for scoring; anything biometric. Single-court footage is explicitly a **no-op** (one court ? everything is "prominent").
22
23   ## 3. Decisions locked
24 | # | Decision | Rationale |
25 |---|----------|-----------|
26 | D1 | The prominent court is a **2D region (polygon) in image space**, refined to a **pseudo-3D play volume** | 1D thresholds don't separate courts under perspective (§1) |
27 | D2 | Gate the **shuttle trajectory** by the region **before windowing** (re-define visibility) | Stops a background rally at the source ? no window ever forms there. Reuses `point_in_polygon` |
28 | D3 | **Default-OFF, flag-gated; single-court = no-op** | Zero-regression rollout ([[weak-signals-retained]]); no behaviour change unless a polygon is supplied + the flag set |
29 | D4 | Reuse the existing **`court_polygon`** config + `point_in_polygon`; do NOT invent a new geometry stack | `FusionConfig.court_polygon` + `fusion_features.point_in_polygon` already exist `[verified]` |
30 | D5 | The prominent court can be **supplied (config) OR auto-derived**; auto-derivation is its own measured step | Config polygon is the cheapest first experiment; auto is the product path |
31
32   ## 4. Design

4.1 The signal change (the core experiment)
Today the windower consumes the raw WASB trajectory: a per-frame `(Frame, Visibility, X, Y)`. The change is a **pre-windowing filter**:

```
for each frame f:  if shuttle_visible(f) and NOT inside_prominent_court(X_f, Y_f):  mark f as NOT visible
```

A background-court shuttle is then indistinguishable from "no shuttle" ? the velocity windower never opens a rally there. This is purely **subtractive** on the shuttle stream (it can only remove visibility, never add it), so on a correctly-drawn prominent court it cannot manufacture rallies ? only suppress out-of-court ones. (Mirror of `rally_gate`'s "do-no-harm" discipline.)

4.2 Identifying the prominent court (four routes, cheapest first)
1. **Configured polygon** (MVP, `[design]?buildable now`): a normalized `[[x,y],?]` court polygon per camera/clip, exactly the existing `FusionConfig.court_polygon`. Draw it once from the dense shuttle/player cluster. Good enough to validate the whole concept on GX010142 + the multicourt golden clips.
2. **Player-location auto-derivation** (`[design]`): run `PersonDetector.detections_per_frame` `[verified] exists`, take the **largest, most-central, most-persistent** player cluster (the foreground players are bigger boxes lower in frame), and fit the prominent-court polygon to where *those* players' feet live. "Prominent = where the match's players are." This is why the owner asked to **extract person features** ? players anchor the court.
3. **Shuttle-density auto-derivation** (`[design]`): the prominent court is the densest sustained shuttle-activity region; cluster the visible trajectory and take the dominant mode. Person-free fallback.
4. **Court-line detection** (`[design]`, heaviest): Hough/segmentation court lines ? the largest court quad. Most accurate, most work; deferred.

v1 ships route 1 (config) + measures; route 2 (player-anchored) is the product upgrade.

4.3 Pseudo-3D play volume (the refinement)
A flat floor polygon rejects a shuttle at the top of a high clear *over the prominent court* (it projects above the floor quad). The pseudo-3D model treats the prominent court as a **floor quad + a vertical air column**: accept a shuttle if its image point lies within the floor polygon **expanded upward** by a height envelope (the max plausible shuttle height projected through the camera). Practically: the floor quad plus an upward `pad` (perspective-aware ? larger near the net/far baseline). Start with a generous fixed upward pad (cheap, MVP); upgrade to a per-camera height envelope only if measurement shows real rallies being clipped. This keeps the floor tight (rejects adjacent-court floor play) while not clipping legitimate high shuttles.

4.4 Reuse map `[verified]`
Need	Existing code
point-in-region test	`backend/pipeline/detectors/fusion_features.py::point_in_polygon`
normalized court polygon config	`backend/config/models.py::FusionConfig.court_polygon` (today: person/Gate-B only)
player boxes per frame	`backend/pipeline/detectors/person_detector.py::PersonDetector.detections_per_frame`
trajectory parse + windowing	`backend/pipeline/detectors/tracknet_runner.parse_trajectory_csv`, `backend/eval/windowing.py`
per-video scoring	`backend/eval/rally_seg_eval.py` (+ the `--stratify` multicourt split, #220)

New code is small: a `prominent_court_gate(points, polygon, height_pad)` that filters trajectory points, threaded into the windowing input behind a default-OFF flag.

4.5 Extension ? deterministic rally-END via shuttle-floor contact (owner note, 2026-06-19) `[design]`

The same court geometry that localizes the prominent court also unlocks a **deterministic rally-END signal** ? the cheapest, most certain way to know a point is over: **the shuttle hit the floor.** This attacks **G3** (rally-END over-extension), where today's shuttle-velocity windowing runs the window long because "shuttle still moving ? rally over."

- **Introduce two geometric primitives:**
  - **The white court lines.** A badminton court is bounded by **white, straight lines forming a rectangle** (with the perspective trapezoid in image space). Detect them ? Hough lines / line-segment detection filtered to white, long, court-consistent segments ? to recover the **court quad** (which also *is* the prominent-court floor polygon of §4.2 route 4; one detector serves both).
  - **The floor plane.** The court lines define the **floor** (the plane the players stand on). The shuttle in flight lives *above* this plane; a shuttle *at* the plane (within the trapezoid, or just outside it) is **on the floor**.
- **The rally-END rule (deterministic):** a rally ends when the shuttle makes **floor contact** ? its trajectory descends to and **stops/rests at the floor level** (a near-zero-velocity terminus at the court-line plane), OR a shuttle that goes to the floor **outside the prominent court** = an out-of-bounds landing (also point-over). Unlike velocity heuristics, a shuttle resting on the floor is **unambiguous** ? the point is over.
- **Why it's strong + cheap:** no new model, no person stack ? it reuses the WASB trajectory (which already gives the shuttle's descending `(x, y)` + the `Visibility` drop when it settles) plus the court-line geometry. It is the **shuttle-side** rally-END signal, complementary to the **player-state** rally-END (player kinematics / held-shuttle) in `RALLY_DETECTION_GAP_CLOSURE.md` §4 Lever A ? fuse both for robustness (a shuttle can settle off-camera; a player can pick up early).
- **Subtlety to measure:** distinguish a true floor landing (point over) from a shuttle that **dips low but is played back up** (a tight net shot / low retrieve). The court-line plane + a brief **dwell** at floor level (the shuttle rests, doesn't re-accelerate up) is the discriminator; tune the dwell window on golden clips.

This is tracked as **C8** below and cross-links gap-closure **G3**. It shares the court-line detector with the prominent-court polygon (§4.2 route 4), so the two land together once court-line detection exists.

5. Experiment plan (measurement)

Land the gate flag-gated, then measure on the **cached trajectories** (CPU, no GPU/re-detect):

1. **GX010142 sanity:** draw a prominent-court polygon; confirm **rally #1 disappears** and the central rallies survive. The litmus the whole idea is born from.
2. **Multicourt golden** (`mahadevpura_2, GX010128, Badminton_BXH_2, Boxhill_Doubles`): ? rally-count, and once those clips are golden-labelled, ? precision / ? recall vs golden via `rally_seg_eval --stratify` (single vs multicourt). The win condition: multicourt precision up, **recall flat** (we drop FPs, not real rallies).
3. **Single-court no-op:** on single-court clips with no polygon (or a full-frame polygon) the gate is a verified no-op ? assert zero change.
4. **Dual-eval discipline** ([[eval-dual-set-regression]]): no per-video regression on either eval set.

Promotion to default-ON is a **separate, owner-gated** decision with a Launch Report (both rulers + the over-seg guard), never bundled with the experiment PR.

6. Risks & limits

- **Polygon is camera-dependent.** A hand-drawn polygon doesn't generalize across setups ? route 2/3 (auto-derivation) is the real product path; v1 config-polygon is for *validation*.
- **Shuttle height** (§4.3): too-tight a floor clips high clears; the pseudo-3D pad mitigates, measurement tunes it.
- **The prominent court can change** (camera re-aim mid-clip): out of scope v1 (assume static per clip).
- **Not a recall fix.** This only removes out-of-court FPs; it does nothing for missed in-court rallies (G1) ? and must be proven not to *cost* recall.

7. Open questions (owner / data)

1. Auto-derivation anchor: **player-location** (route 2) vs **shuttle-density** (route 3) as the default once validated? *(decides: data, post-experiment)*
2. Height-envelope: fixed upward pad vs per-camera perspective model ? is the fixed pad

enough? *(decides: measurement)*
95   3. Do we expose the prominent court as a **one-time per-camera setup** (draw-once UI) or
always auto? *(decides: owner/product)*
96
97   ## 8. Deliverables & progress tracker ? source of truth
98   Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, off
`origin/master`, no stacking.
99
100 | ID | Deliverable | Depends on | Gated? | Status | PR |
101 |----|-------------|-----------|--------|--------|----|
102 | C0 | This design doc | ? | No | ? | this PR |
103 | C1 | `prominent_court_gate(points, polygon, height_pad)` + unit tests (pure, point-in-
polygon) | ? | No (default-OFF) | ? | ? |
104 | C2 | Thread the gate into the windowing input behind a default-OFF flag
(`court_polygon` + `gate_shuttle_to_court`) | C1 | No (default-OFF) | ? | ? |
105 | C3 | GX010142 litmus: rally #1 dropped, central rallies kept (measured) | C2 | No | ?
| ? |
106 | C4 | Extract person features (`detections_per_frame`) on GX010142 ? player-cluster =
prominent court; measure `shuttle_player_colocation` too | ? | No | ? | ? |
107 | C5 | Multicourt-golden measurement (? count now; ? precision/recall vs golden once
labelled) | C2, labels | ? owner labels | ? | ? |
108 | C6 | Auto-derive the polygon (route 2 player-anchored) | C4 | ? owner | ? | ? |
109 | C7 | Launch decision (default-ON) with a Launch Report | C3,C5 | ? owner | ? | ? |
110 | C8 | **Court-line / floor detection** ? (a) auto court polygon, (b) **deterministic
shuttle-floor-contact rally-END** (§4.5, targets G3) | ? | ? owner | ? | ? |
111
112   ## 9. Definition of done
113   - [ ] The gate exists, is **default-OFF**, unit-tested, and a verified no-op on single-
court / no-polygon.
114   - [ ] GX010142 rally #1 (and the far-right adjacent-court band) is dropped while the
central rallies survive `[measured]`.
115   - [ ] On the multicourt golden clips, multicourt **precision improves with recall flat**
vs golden labels (no per-video regression).
116   - [ ] A winning configuration is flipped default-ON only via a Launch Report ? flip
`Status ? DONE`, archive.
117
118   ## 10. References
119   **Code `[verified]`:** `fusion_features.py` (`point_in_polygon`, `person_in_court`),
`person_detector.py` (`detections_per_frame`), `config/models.py`
(`FusionConfig.court_polygon`), `eval/windowing.py`, `eval/rally_seg_eval.py` (`--stratify`).
120   **Docs:** [`RALLY_DETECTION_GAP_CLOSURE.md`](RALLY_DETECTION_GAP_CLOSURE.md) (G2),
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md) (Gate B / court mask),
[`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md),
[`QUALITY_ITERATIONS.md`](QUALITY_ITERATIONS.md).
121
122   ---
123
124   ### Changelog
125   - 2026-06-19 ? Created from the owner's GX010142 rally-#1 (background-court FP)
observation + the measured multicourt over-fire (G2). Validated the 2D/pseudo-3D need from the
cached trajectory; mapped the reuse onto the existing `court_polygon` + `point_in_polygon`
machinery.
126   - 2026-06-19 ? Added §4.5 (owner note): a **deterministic rally-END signal via shuttle-
floor contact** ? detect the white court lines ? the floor plane ? a shuttle resting on the
floor (or landing out-of-court) ends the rally (targets G3). Shares the court-line detector with
the prominent-court polygon; tracked as C8.
127

24. assistant (2026-06-19T16:57:55.077Z)

The doc confirms (c) is explicitly DESIGN-ONLY. §4.5 is tagged `[design]`, and the progress tracker C8 (which covers "deterministic shuttle-floor-contact rally-END") is status ? (Todo), gated, with no PR. The `court_gate.py` floor matches are about the prominent-court air-column polygon test, not a rally-END floor-contact detector. Let me confirm there's no floor-contact rally-END implementation anywhere and verify the default segmenter isn't fusion_hybrid. I'll

also double-check whether the served Gate-A in fusion_hybrid is reachable from the default pipeline.

25. user (2026-06-19T16:57:57.216Z)

No files found

26. user (2026-06-19T16:57:57.519Z)

```
1       """Prominent-court shuttle gate ? multicourt G2 (see
``docs/PROMINENT_COURT_DETECTION.md``).
2
3       In multicourt footage the WASB shuttle tracker follows ANY shuttle in frame, so a rally
on a
4       background / adjacent court becomes a false rally on the filmed match. This gate re-
defines the
5       shuttle signal ``(x, y, visible)`` ? ``(x, y, visible AND inside-the-prominent-court)``:
a visible
6       shuttle point whose normalised position is outside the prominent-court polygon is marked
NOT
7       visible, so the velocity windower never opens a rally there.
8
9       Pure (no GPU / cv2): point-in-polygon over the court polygon (reuses
10      ``fusion_features.point_in_polygon``). **Subtractive only** ? it can only CLEAR
visibility, never
11      set it ? so on a correct polygon it suppresses out-of-court rallies without
manufacturing any.
12      ``polygon=None`` (or < 3 vertices, or non-positive frame dims) ? an EXACT no-op (single-
court /
13      unconfigured footage), matching ``rally_gate``'s do-no-harm discipline.
14
15      The polygon is the court FLOOR in normalised 0-1 image coords (same convention as
16      ``FusionConfig.court_polygon``). ``height_pad`` is the MVP pseudo-3D play-volume (§4.3):
a point up
17      to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is
still
18      admitted, so a high clear over the prominent court is not clipped.
19      """
20      from __future__ import annotations
21
22      from dataclasses import replace
23      from typing import Any, List, Optional, Sequence, Tuple
24
25      from backend.pipeline.detectors.fusion_features import point_in_polygon
26
27      Polygon = Sequence[Tuple[float, float]]
28
29
30      def admitted(nx: float, ny: float, polygon: Polygon, height_pad: float = 0.0) -> bool:
31          """Is the normalised point inside the court floor polygon, OR within ``height_pad``
above it?
32
33          The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the
floor) and
34          finding it inside means the original sits within ``height_pad`` above the floor ? a
high shuttle
35          over the prominent court, admit it."""
36          if point_in_polygon((nx, ny), polygon):
37              return True
38          return height_pad > 0.0 and point_in_polygon((nx, ny + height_pad), polygon)
39
40
41      def in_court_fraction(interval: Tuple[float, float], points: List[Any], polygon:
Polygon,
42                            fps: float, frame_width: float, frame_height: float,
```

```
                           height_pad: float = 0.0) -> float:
        """Fraction of a candidate rally's VISIBLE shuttle points that lie inside the
prominent court.

        ``interval`` is ``(start_s, end_s)``; points carry ``.frame``. Returns 1.0 when
there are no
        visible points in the interval (a window with no shuttle is not an out-of-court
false positive ?
        leave that judgement to the windower)."""
        if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
            return 1.0
        fs, fe = interval[0] * fps, interval[1] * fps
        n = inside = 0
        for p in points:
            if getattr(p, "visible", False) and fs <= p.frame <= fe:
                n += 1
                if admitted(p.x / frame_width, p.y / frame_height, polygon, height_pad):
                    inside += 1
        return 1.0 if n == 0 else inside / n


    def gate_intervals_to_court(intervals: List[Tuple[float, float]], points: List[Any],
                                polygon: Optional[Polygon], fps: float,
                                frame_width: float, frame_height: float,
                                min_in_court_frac: float = 0.5, height_pad: float = 0.0
                                ) -> List[Tuple[float, float]]:
        """Drop candidate rallies whose shuttle is MAJORITY out-of-court ? the window-level
gate.

        Point-level gating (:func:`gate_points_to_court`) is leaky: a background-court rally
whose shuttle
        occasionally drifts into the prominent court keeps a few in-court points, enough to
hold a (sparse)
        window (measured: GX010142 rally #1 is 90% adjacent-court yet a 10% central tail
survives a hard
        point gate). This drops an interval whose in-court point fraction is below
``min_in_court_frac``,
        so a rally that is *mostly* in the adjacent court is removed wholesale. No-op
(returns all
        intervals) when ``polygon`` is None/degenerate. Subtractive-only."""
        if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
            return intervals
        return [iv for iv in intervals
                if in_court_fraction(iv, points, polygon, fps, frame_width, frame_height,
height_pad)
                >= min_in_court_frac]


    def gate_points_to_court(points: List[Any], polygon: Optional[Polygon],
                             frame_width: float, frame_height: float,
                             height_pad: float = 0.0) -> List[Any]:
        """Return ``points`` with visibility CLEARED for any visible shuttle outside the
prominent-court
        polygon (expanded upward by ``height_pad``).

        No-op (returns the input list unchanged) when ``polygon`` is None/degenerate or the
frame dims
        are non-positive. Otherwise each gated point is a NEW ``TrajectoryPoint`` (the input
list and
        its points are never mutated)."""
        if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0:
            return points
        out: List[Any] = []
```

```
93          for p in points:
94              if getattr(p, "visible", False):
95                  nx, ny = p.x / frame_width, p.y / frame_height
96                  if not admitted(nx, ny, polygon, height_pad):
97                      p = replace(p, visible=False)
98              out.append(p)
99          return out
100
```

27. assistant (2026-06-19T16:58:05.169Z)

This confirms (c): `court_gate.py` is the **prominent-court spatial gate** (G2 / shuttle localization), which IS built. But the shuttle-floor-contact **rally-END** signal (§4.5 / C8) ? detecting floor contact to end a rally ? is NOT implemented. The "floor" references in court_gate.py are the court-floor *polygon* used for spatial gating, not a floor-contact terminus detector. No `floor_contact`/rally-END-via-floor code exists.

Now let me confirm the prominent-court window gate (e358b28, PR #243) is wired into production but default-OFF, and confirm the default segmenter. Let me check where `court_gate` is consumed in the production segmenter path and how `gate_shuttle_to_court` / config flag is wired.

28. user (2026-06-19T16:58:07.595Z)

```
Found 2 files
backend\eval\rally_seg_eval.py
backend\pipeline\detectors\court_gate.py
```

29. user (2026-06-19T16:58:09.188Z)

```
backend\pipeline\segmenters\motion.py:255:segmenter_registry.register(MotionPlaySegmenter)
backend\pipeline\segmenters\gemini.py:559:segmenter_registry.register(GeminiPlaySegmenter)
backend\pipeline\segmenters\fusion_hybrid.py:4:TrackNet optional) to seed candidate windows
exactly as `wasb_hybrid`/`tracknet_hybrid`
backend\pipeline\segmenters\fusion_hybrid.py:24:and hands off identically to `wasb_hybrid`.
Registered as ``fusion_hybrid``, NOT the default
backend\pipeline\segmenters\fusion_hybrid.py:176:segmenter_registry.register(FusionSegmenter)
backend\pipeline\segmenters\tracknet_hybrid.py:27:        return "tracknet_hybrid"
backend\pipeline\segmenters\tracknet_hybrid.py:42:segmenter_registry.register(TrackNetHybridSegm
enter)
backend\pipeline\segmenters\base.py:39:    def register(self, segmenter_cls:
Type[BasePlaySegmenter]):
backend\pipeline\segmenters\wasb_hybrid.py:29:        return "wasb_hybrid"
backend\pipeline\segmenters\wasb_hybrid.py:41:segmenter_registry.register(WasbHybridSegmenter)
backend\pipeline\segmenters\__init__.py:5:from backend.pipeline.segmenters.tracknet_hybrid
import TrackNetHybridSegmenter
backend\pipeline\segmenters\__init__.py:6:from backend.pipeline.segmenters.wasb_hybrid import
WasbHybridSegmenter
backend\pipeline\segmenters\yolo_hybrid.py:15:The registry key stays "yolo_hybrid" for back-
compat with existing configs and the
backend\pipeline\segmenters\yolo_hybrid.py:50:        return "yolo_hybrid"
backend\pipeline\segmenters\yolo_hybrid.py:250:                    video_id,
detector="yolo_hybrid",
backend\pipeline\segmenters\yolo_hybrid.py:436:segmenter_registry.register(HybridYoloSegmenter)
backend\pipeline\segmenters\trajectory_hybrid.py:3:tracknet_hybrid and wasb_hybrid were copy-
paste twins ? CODE_MAP carried a
backend\pipeline\segmenters\trajectory_hybrid.py:81:    # --- Fusion seams (identity by default
? wasb_hybrid/tracknet_hybrid unchanged) -------
```

30. assistant (2026-06-19T16:58:13.465Z)

Critical finding: `court_gate.py` (the prominent-court gate from PR #243) is consumed ONLY in `backend/eval/rally_seg_eval.py` ? it is NOT imported by any production segmenter under `backend/pipeline/segmenters/`. So the prominent-court spatial gate is **eval-only**, not wired into the production indexing pipeline. This matters because the design doc §4.5 builds on it.

Let me confirm court_gate is not referenced in the production pipeline and verify how rally_seg_eval uses it.

31. user (2026-06-19T16:58:15.164Z)

```
194-    poly = video.get("court_polygon")
195-    if poly and ivs:
196:        from backend.pipeline.detectors.court_gate import gate_intervals_to_court
197-        fw = float(video["frame_width"])
198-        fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
199:        ivs = gate_intervals_to_court(
200-            ivs, points, poly, float(video["fps"]), fw, fh,
201-            min_in_court_frac=float(video.get("court_min_in_court_frac", 0.5)),
```

32. user (2026-06-19T16:58:16.420Z)

No matches found

33. assistant (2026-06-19T16:58:20.407Z)

Confirmed. `court_gate` (prominent-court gate) is consumed only inside `backend/eval/rally_seg_eval.py` ? it is NOT wired into any production segmenter. This is relevant context but slightly tangential to the four specific levers asked. Let me note it where relevant.

Now I have everything I need to render the verdicts. Let me confirm the fusion_hybrid IS importable/registered in the pipeline (it self-registers at line 176, and is imported via `__init__.py`). Let me quickly confirm fusion_hybrid is imported in segmenters `__init__`.

34. user (2026-06-19T16:58:21.078Z)

```
1       from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2       from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3       from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4       from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6       from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7       from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
8
9       __all__ = [
10          "segmenter_registry",
11          "BasePlaySegmenter",
12          "MotionPlaySegmenter",
13          "GeminiPlaySegmenter",
14          "HybridYoloSegmenter",
15          "TrackNetHybridSegmenter",
16          "WasbHybridSegmenter",
17          "FusionSegmenter",
18      ]
19
```

35. assistant (2026-06-19T16:58:35.513Z)

I now have full verification. Let me synthesize the verdicts.

Summary of build status for the four levers the design reuses:

(a) rally_gate.py Gate-A net-crossings ? BUILT, and WIRED into the `fusion_hybrid` segmenter (production code, registered + imported), but `fusion_hybrid` is NOT the default segmenter (`default_segmenter="yolo_hybrid"`), and the served gate is default-OFF (`min_crossings=0`). The net-crossing count is always *persisted* as a feature; *gating* the handoff is opt-in. So it's wired into a real segmenter but does not affect the default pipeline / no default gating. Best label: BUILT, wired into the (opt-in, non-default) fusion segmenter as a persisted feature; served gating default-OFF ? effectively eval/experiment-grade, not active in the default prod

path.

(b) R4 `_trim_inactive_tail` ? BUILT in tracknet_runner.py, wired into `trajectory_to_action_windows` (used by all trajectory-hybrid segmenters), and DEFAULT-ON (`window_inactivity_end=1.5`, the R1 milestone). This is the one lever that is actually live in production by default.

(c) shuttle-floor-contact rally-END (§4.5) ? DESIGN-ONLY. Tagged `[design]`, tracked as C8 (Todo/gated, no PR). No floor-contact terminus code exists. The `court_gate.py` "floor" references are the spatial prominent-court polygon (G2), a different lever.

(d) shuttle_player_colocation ? BUILT in fusion_features.py, wired into fusion_hybrid's person path, but only computed when the person cue is enabled (`cue_flags.person`, default OFF). Persisted feature when on; never served-gated on directly.

36. user (2026-06-19T16:58:57.295Z)

Structured output provided successfully

37. assistant (2026-06-19T16:59:08.797Z)

Verification complete. Findings (all from the actual repo-root files, worktree copies ignored):

(a) rally_gate.py Gate-A net-crossings ? BUILT, wired to the opt-in fusion segmenter only; served gating default-OFF. `backend/pipeline/detectors/rally_gate.py:91-117` implements it; `backend/pipeline/segmenters/fusion_hybrid.py:35,83-85` invokes it to persist `net_crossings` per window, and `:141-173` can gate the handoff. But `fusion_hybrid` is NOT the default segmenter (`backend/config/models.py:118` default is `yolo_hybrid`; the file self-documents "NOT the default segmenter ? it is opt-in"), and `min_crossings` defaults to 0 (`models.py:105`), so it persists a feature but does not gate by default. So: not eval-only (it lives in a real segmenter), but not active in the default prod path either.

(b) R4 `_trim_inactive_tail` ? BUILT+WIRED, and default-ON. `tracknet_runner.py:401-428`, invoked in `trajectory_to_action_windows:379-382` (the shared windowing path). Config default `window_inactivity_end=1.5` (`models.py:153`, the R1 milestone) means it is live in production by default ? the only one of the four that is.

(c) shuttle-floor-contact rally-END (§4.5) ? DESIGN-ONLY.
`docs/PROMINENT_COURT_DETECTION.md:65` is tagged `[design]`; tracker row C8 (`:110`) is Todo/owner-gated, no PR. No `floor_contact`/floor-plane/court-line/dwell code exists. The "floor" in `court_gate.py` is the prominent-court *spatial* polygon (G2), a different already-built lever.

(d) shuttle_player_colocation ? BUILT, person-cue default-OFF. `fusion_features.py:168-189`, in `PERSON_FEATURE_NAMES`, computed by `compute_person_features`. Wired in `fusion_hybrid.py:88-139` but only when `cue_flags.person` is set, which defaults OFF (`models.py:102`). Persisted feature when on; never served-gated on directly.

One adjacent note worth flagging for the design doc: the prominent-court spatial gate (`court_gate.py`, PR #243) that §4.5 builds on is itself consumed only by `backend/eval/rally_seg_eval.py:196-201` ? it is NOT imported by any production segmenter, so that reused lever is eval-only too.